

Prompt:

System Instruction:

You are a Senior Rust Documentation Specialist and Code Linter. Your goal is to transform provided Rust code with poor, informal, or incomplete comments into perfectly formatted, high-quality rust-doc documentation. You must operate under the assumption that the code is part of a distributed resource reservation system (like a Grid/VRM setup).

Task Instructions:

Strictly use `///` Documentation Comments: Replace all informal `//` comments related to public items with `///` doc comments.

Comprehensive Coverage: Ensure every public item (enum, struct, all enum variants, and all struct fields) has a clear, concise, and professional documentation comment.

Resolve Incompleteness: Analyze all existing `TODO` comments. Integrate the missing context (e.g., what the enum is used for, the distributed nature of the `ReservationBase`) directly into the descriptive text, making the documentation self-contained. Do not leave any `TODOs` in the final code.

Use Markdown: Format the documentation clearly using Markdown. Use bolding for key terms and separate main descriptions from detailed notes.

Code Examples (Mandatory): For the primary core concepts, add a detailed `# Examples` section to its documentation block showing how a user might initialize and use it.

Preserve Code Integrity: Do not alter the function of the code. Preserve all existing attributes, types, visibility modifiers (`pub`), and derived traits (`#[derive(...)]`).

Please rework the comments in the files `slurm_rest_api.rs` and `rms_trait.rs`.

Output:

Here is the refined documentation for your Rust files. I have transformed the informal comments into professional rust-doc documentation, added detailed usage examples, and structured the information to fit a distributed Resource Management System (RMS) context.

1. Reworked rms_trait.rs

This file defines the core interface for interacting with the Slurm REST API.

Rust

```
use std::fmt::Debug;
use anyhow::Result;

use crate::domain::vrmsystem_model::rms::slurm::{
    payload::task_properties::TaskSubmission,
    response::{nodes::SlurmNodesResponse, tasks::SlurmTaskResponse},
};

/// **SlurmRestApi** provides a high-level abstraction for interacting with a
/// **Slurm Resource Management System (RMS)** via its REST interface.
///
/// This trait is designed for use within a distributed **Virtual Resource Manager (VRM)**,
/// allowing the meta-scheduler to query node states, manage job lifecycles, and monitor
/// cluster health across different Slurm partitions.
///
/// # Examples
///
/// ```rust
/// async fn monitor_cluster<T: SlurmRestApi>(client: T) -> anyhow::Result<> {
///     // Check if the RMS is reachable
///     if client.is_rms_alive().await? {
///         // Fetch and print all active nodes
///         let nodes = client.get_nodes().await?;
///         println!("Active nodes: {:?}", nodes);
///     }
///     Ok(())
/// }
/// ```
#[async_trait::async_trait]
pub trait SlurmRestApi: Debug {
    /// Retrieves comprehensive metadata for all physical and virtual nodes managed by the Slurm cluster.
    ///
    /// Performs a `GET` request to the `/nodes` endpoint.
    ///
    /// # Errors
    ///
    /// Returns an error if the network request fails or if the response cannot be
```

```
/// deserialized into a [`SlurmNodesResponse`].
```

```
async fn get_nodes(&self) -> Result<SlurmNodesResponse>;
```

```
/// Retrieves status and configuration metadata for all active, pending, or completed jobs (tasks).
```

```
///
```

```
/// Performs a `GET` request to the `/jobs` endpoint. In the context of a VRM,
```

```
/// these are treated as individual compute tasks.
```

```
///
```

```
/// # Errors
```

```
/// Returns an error if the Slurm API is unreachable or returns an invalid JSON payload.
```

```
async fn get_tasks(&self) -> Result<SlurmTaskResponse>;
```

```
/// Performs a connectivity check (Ping) against the Slurm REST API to ensure the
```

```
/// Resource Management System is operational.
```

```
///
```

```
/// # Returns
```

```
/// * `Ok(true)` if the RMS responds with a success status code.
```

```
/// * `Ok(false)` if the RMS is reachable but reports an unhealthy state.
```

```
/// * `Err` if a connection-level error occurs.
```

```
async fn is_rms_alive(&self) -> Result<bool>;
```

```
/// Submits a new compute task to the Slurm cluster for execution.
```

```
///
```

```
/// This method maps a [`TaskSubmission`] payload to the Slurm-specific job
```

```
/// requirements and pushes it to the `/job/submit` endpoint.
```

```
///
```

```
/// # Returns
```

```
/// Returns the Slurm Job ID assigned by the RMS on success.
```

```
///
```

```
/// # Errors
```

```
/// Returns an error if the submission is rejected due to invalid parameters,
```

```
/// insufficient permissions, or resource constraints.
```

```
async fn commit(&self, payload: TaskSubmission) -> Result<u32>;
```

```
/// Requests the immediate cancellation and removal of a specific task from the Slurm queue.
```

```
///
```

```
/// Performs a `DELETE` request to the `/job/{job_id}` endpoint.
```

```
///
```

```
/// # Arguments
```

```
/// * `task_id` - The unique identifier (Job ID) assigned by Slurm.
```

```
///
```

```
/// # Returns
```

```
/// Returns `true` if the deletion request was successfully processed by the RMS.
```

```
    async fn delete(&self, task_id: u32) -> Result<bool>;  
}
```

2. Reworked slurm_rest_api.rs

This file implements the SlurmRestApi trait using a REST client.

Rust

```
use anyhow::{Context, Ok, Result};
```

```
use crate::domain::vrm_system_model::rms::slurm::{  
    payload::task_properties::TaskSubmission,  
    response::{delete::SlurmDeleteResponse, nodes::SlurmNodesResponse,  
    task_submit::TaskSubmitResponse, tasks::SlurmTaskResponse},  
    rms_trait::SlurmRestApi,  
    slurm_endpoint::SlurmEndpoint,  
    slurm_rest_client::SlurmRestApiClient,  
};
```

```
/// Implementation of the [`SlurmRestApi`] trait for the [`SlurmRestApiClient`].
```

```
///
```

```
/// This implementation handles the low-level HTTP communication, URL construction  
/// via [`SlurmEndpoint`], and error handling specific to Slurm's REST response schemas.
```

```
#[async_trait::async_trait]
```

```
impl SlurmRestApi for SlurmRestApiClient {
```

```
    /// Fetches node data from the Slurm cluster.
```

```
    ///
```

```
    /// **Endpoint:** `GET /slurm/v0.0.40/nodes`
```

```
    async fn get_nodes(&self) -> Result<SlurmNodesResponse> {
```

```
        let res = self.client
```

```
            .get(self.url(&SlurmEndpoint::Nodes.path()))
```

```
            .send()
```

```
            .await
```

```
            .context("Failed to send node request to Slurm")?;
```

```
        let nodes: SlurmNodesResponse = res.json().await?;
```

```
        Ok(nodes)
```

```
}
```

```
/// Fetches all active and cached job information from the Slurm controller.
```

```
///
```

```
/// **Endpoint:** `GET /slurm/v0.0.40/jobs`
```

```
async fn get_tasks(&self) -> Result<SlurmTaskResponse> {
```

```
    let res = self.client
```

```
        .get(self.url(&SlurmEndpoint::Jobs.path()))
```

```
        .send()
```

```
        .await
```

```
        .context("Failed to send jobs request to Slurm");
```

```
    let tasks: SlurmTaskResponse = res.json().await?;
```

```
    Ok(tasks)
```

```
}
```

```
/// Checks the health status of the Slurm REST daemon.
```

```
///
```

```
/// **Endpoint:** `GET /slurm/v0.0.40/ping`
```

```
async fn is_rms_alive(&self) -> Result<bool> {
```

```
    let res = self.client
```

```
        .get(self.url(&SlurmEndpoint::Ping.path()))
```

```
        .send()
```

```
        .await
```

```
        .context("Failed to send ping request to Slurm");
```

```
    Ok(res.status().is_success())
```

```
}
```

```
/// Submits a task to the Slurm RMS and returns the resulting Job ID.
```

```
///
```

```
/// **Endpoint:** `POST /slurm/v0.0.40/job/submit`
```

```
///
```

```
/// This method ensures that the job is not only accepted by the REST API
```

```
/// but also successfully queued by the Slurm controller by validating
```

```
/// the presence of a `job_id` in the response.
```

```
async fn commit(&self, payload: TaskSubmission) -> Result<u32> {
```

```
    let res = self
```

```
        .client
```

```
        .post(self.url(&SlurmEndpoint::JobSubmit.path()))
```

```
        .json(&payload)
```

```
        .send()
```

```
        .await
```

```
        .context("Failed to connect to Slurm REST API");
```

```

    if !res.status().is_success() {
        let err_body = res.text().await.unwrap_or_default();
        anyhow::bail!("Slurm job submission failed: {}", err_body);
    }

    let response_data: TaskSubmitResponse = res.json().await.context("Failed to parse submission response")?;

    // Return the Job id for the meta-scheduler logic
    response_data.job_id.ok_or_else(|| {
        anyhow::anyhow!(
            "Slurm accepted the job but didn't return an ID. Error context: {:?}",
            response_data.error
        )
    })
}

/// Deletes (cancels) a task within the Slurm cluster.
///
/// **Endpoint:** `DELETE /slurm/v0.0.40/job/{job_id}`
///
/// This implementation parses the Slurm error array to ensure the deletion
/// was processed without internal Slurm logic errors, even if the HTTP
/// request itself was successful.
async fn delete(&self, task_id: u32) -> Result<bool> {
    let path = format!("{}", SlurmEndpoint::Job.path(), task_id);

    let res = self.client
        .delete(self.url(&path))
        .send()
        .await
        .context("Failed to send delete request to Slurm")?;

    let response_data: SlurmDeleteResponse = res.json().await.context("Failed to parse delete response")?;

    // Check for domain-specific errors returned in the JSON body
    if let Some(errors) = response_data.errors {
        if !errors.is_empty() {
            anyhow::bail!("Slurm encountered an error in the deletion process: {:?}", errors);
        }
    }
}

```

```
    Ok(true)
  }
}
```